JavaScript Arrays

Quick-Revision Sheet | 20 key questions, each with code and the concept it teaches

How to use this sheet

1. Read the **Question**, cover the Code column and try to solve it yourself.
2. Compare with the **Code**, then read the **Key Concept** - that is the idea to remember, so you never relearn it.
3. Tick the **checkbox** under each question when you've mastered it (works in Chrome, Edge, Acrobat). Click any section below to jump to it.

CONTENTS	WHAT YOU WILL LEARN	QUESTIONS
Section A - Basics & Mutation	Indexing, add/remove, splice vs slice, sorting, searching. Know what changes the original array.	Q1 - Q6
Section B - Iteration & Higher-Order Methods	The callback methods you will use daily: forEach, map, filter, find, some/every, reduce.	Q7 - Q12
Section C - Modern Syntax & References	Destructuring, spread, rest, and the reference behavior that causes most array bugs.	Q13 - Q17
Section D - Challenge Problems	Combine everything: nested arrays, custom comparison, and a full method chain.	Q18 - Q20
Section E - Cheat Sheet	Which methods mutate? Which method for which goal?	1 page

Section A - Basics & Mutation

Indexing, add/remove, splice vs slice, sorting, searching. Know what changes the original array.

QUESTION	CODE	KEY CONCEPT - WHAT TO REMEMBER

Q1 ☐ *Done*
Create, access, update, length
Update an item, get the last item without hardcoding the index, then shrink the array.
	`let langs = ["JS", "Python", "Java", "C++"];`
	`langs[2] = "TypeScript";  // update by index`
	`langs[langs.length - 1];  // "C++"  (classic)`
	`langs.at(-1);  // "C++"  (modern)`
	`langs.length = 2;  // truncates -> ["JS", "Python"]`
		• Arrays are **zero-indexed**; last index = `length - 1`.
		• `.at(-1)` counts from the end - cleaner for the last item.
		• `length` is writable: shrinking it **deletes** items from the end.
		• Assigning to an index **mutates** the array.

Q2 ☐ *Done*
Queue with push / pop / shift / unshift
Add and remove people from both ends and capture the removed names.
	`let queue = ["Aman", "Riya"];`
	`queue.push("Karan");  // add at end`
	`queue.unshift("Sarthak");  // add at start`
	`let served    = queue.shift(); // removes first -> "Sarthak"`
	`let cancelled = queue.pop();   // removes last  -> "Karan"`
	`// queue is now ["Aman", "Riya"]`
		• `push/pop` work at the **end**; `unshift/shift` at the **start**.
		• Adders return the **new length**; removers return the **removed item**.
		• All four **mutate** the original array.

Q3 ☐ *Done*
Remove / insert / replace with splice
Remove two items, insert one without deleting, replace one - all with splice.
	`let a = ["HTML","CSS","JavaScript","React","Node.js"];`
	`a.splice(2, 2);  // remove 2 items from index 2`
	`  // -> ["HTML","CSS","Node.js"]`
	`let r = ["HTML", "CSS", "React"];`
	`r.splice(2, 0, "JavaScript");  // insert (deleteCount = 0)`
	`r.splice(1, 1, "Tailwind");  // replace "CSS"`
		• `splice(start, deleteCount, ...newItems)` does **remove, insert and replace**.
		• `deleteCount = 0` means pure **insert**.
		• It **mutates** and returns an array of the removed items.

Q4 ☐ *Done*
slice vs splice
Copy [20,30,40] without touching the array, then actually cut it out.
	`let arr = [10, 20, 30, 40, 50];`
	`let part = arr.slice(1, 4);  // [20,30,40]  arr unchanged`
	`let cut  = arr.splice(1, 3);  // [20,30,40]  arr -> [10, 50]`
		• `slice(start, end)` - **end is excluded**, does **not** mutate (photocopy).
		• `splice(start, count)` - **mutates** the array (cutting pages out).
		• `arr.slice()` with no arguments = quick shallow copy.

Q5 ☐ *Done*
Numeric sorting
Sort prices ascending and descending. Why is plain sort() unreliable?
	`let prices = [499, 99, 1299, 299, 999];`
	`[...prices].sort((a, b) => a - b);  // [99,299,499,999,1299]`
	`[...prices].sort((a, b) => b - a);  // [1299,999,499,299,99]`
	`prices.sort();  // WRONG -> [1299, 299, 499, 99, 999]`
		• Default `sort()` compares as **strings** (so 1299 sorts before 299).
		• Compare fn: negative -> `a` first, positive -> `b` first.
		• `sort()` **mutates** - copy with spread first (or use `toSorted()`).

Q6 ☐ *Done*
Search & join
Check existence, find positions, and turn the array into a string.
	`let skills = ["HTML", "CSS", "JavaScript", "React"];`
	`skills.includes("JavaScript");  // true`
	`skills.indexOf("React");  // 3`
	`skills.indexOf("Node.js");  // -1  (not found)`
	`skills.join(" -> ");  // "HTML -> CSS -> JavaScript -> React"`
		• `includes` -> **boolean**; `indexOf` -> **position or -1**.
		• `join(separator)` converts an array to a **string**.
		• All three are **non-mutating**.

Section B - Iteration & Higher-Order Methods

The callback methods you will use daily: forEach, map, filter, find, some/every, reduce.

QUESTION	CODE	KEY CONCEPT - WHAT TO REMEMBER

Q7 ☐ *Done*
forEach and its return value
Print each student's mark. What does forEach itself return?
	`let marks = [45, 67, 89, 32, 76];`
	`let res = marks.forEach((mark, i) => {`
	`  console.log(`Student ${i + 1}: ${mark}`);`
	`});`
	`console.log(res);  // undefined`
		• **`forEach`** is for **side effects** (log, DOM, send data).
		• It always returns **`undefined`** - cannot be chained.
		• Callback receives **`(value, index, array)`**.

Q8 ☐ *Done*
map - transform every item
Add 18% GST to every price without changing the original.
	`let prices = [100, 250, 500, 1000];`
	`let withGST = prices.map(p => p * 1.18);`
	`// [118, 295, 590, 1180]  prices is unchanged`
		• **`map`** returns a **new array of the same length**.
		• The callback **must return** the new value.
		• For money, beware floating point - use **`Math.round`** / **`toFixed`**.

Q9 ☐ *Done*
filter - keep matching items
Keep only scores >= 40, then only scores >= 80.
	`let scores = [23, 45, 67, 34, 89, 90, 12];`
	`scores.filter(s => s >= 40);  // [45, 67, 89, 90]`
	`scores.filter(s => s >= 80);  // [89, 90]`
		• Callback returns **true -> keep**, false -> drop.
		• Result can be **shorter** (or **`[]`**); original untouched.

Q10 ☐ *Done*
find & findIndex
First adult user; position of the first price above 500; what if nothing matches?
	`let users = [{ name: "Aman", age: 17 }, { name: "Riya", age: 19 }];`
	`users.find(u => u.age >= 18);  // { name:"Riya", age:19 }`
	`users.find(u => u.age > 99);  // undefined`
	`[199, 499, 799].findIndex(p => p > 500);  // 2`
	`[199, 499, 799].findIndex(p => p > 5000);  // -1`
		• **`find`** -> first matching **item**, else **`undefined`**.
		• **`findIndex`** -> its **index**, else **`-1`**.
		• Both stop at the **first** match (unlike **`filter`**).

Q11 ☐ *Done*
some & every
Is anyone under 18? Is everyone 18+? Is every age positive?
	`let ages = [19, 22, 17, 24, 30];`
	`ages.some(a => a < 18);  // true  (at least one)`
	`ages.every(a => a >= 18);  // false  (all must pass)`
	`ages.every(a => a > 0);  // true`
		• **`some`** = **OR**, **`every`** = **AND**; both return a boolean.
		• Both **short-circuit** as soon as the answer is known.
		• Empty array: **`some`** -> false, **`every`** -> **true**.

Q12 ☐ *Done*
reduce - many values into one
Calculate the cart total, then apply a flat discount of 500.
	`let cart = [499, 1299, 299, 799];`
	`let total = cart.reduce((acc, price) => acc + price, 0);  // 2896`
	`let final = total - 500;  // 2396`
		• **`reduce((acc, item) => newAcc, initialValue)`**.
		• **Always pass an initial value** (**`0`** here).
		• Callback must **return the new accumulator**.
		• Use for sums, counts, max, grouping.

Section C - Modern Syntax & References

Destructuring, spread, rest, and the reference behavior that causes most array bugs.

QUESTION	CODE	KEY CONCEPT - WHAT TO REMEMBER

Q13 ☐ *Done*
Destructuring and swapping
Pull values by position, skip one, and swap two variables with no temp variable.
	`let profile = ["Sarthak", 23, "Developer", "Bhopal"];`
	`let [name, , profession, city] = profile;  // empty slot skips age`
	`let [x, y = 0] = [5];  // default value -> y = 0`
	`let first = "frontend", second = "backend";`
	`[first, second] = [second, first];  // swap`
		• Destructuring assigns **by position**.
		• Skip with an empty slot; give a **default** with **=**.
		• **[a, b] = [b, a]** is the one-line swap.
		• Put a **;** before a line that starts with **[**.

Q14 ☐ *Done*
Spread - copy and merge
Copy an array, add to the copy, and merge two arrays.
	`let fe = ["HTML", "CSS", "JavaScript"];`
	`let be = ["Node.js", "Express"];`
	`let copy = [...fe];  // new top-level copy`
	`copy.push("React");  // fe is unchanged`
	`let fullStack = [...fe, ...be];  // merge`
	`let added = [...fe, "Vue"];  // non-mutating add`
		• **...** **expands** items into a new array.
		• Use for **copy, merge, and non-mutating add**.
		• The backbone of immutable updates (React state).

Q15 ☐ *Done*
Reference vs copy
Predict the output. Why is arr1 === arr2 true but arr1 === arr3 false?
	`let arr1 = [1, 2, 3];`
	`let arr2 = arr1;  // same reference`
	`let arr3 = [...arr1];  // new array`
	`arr2.push(4);  // arr1 & arr2 -> [1,2,3,4]`
	`arr3.push(5);  // arr3 -> [1,2,3,5]`
	`arr1 === arr2;  // true`
	`arr1 === arr3;  // false  (same values, different object)`
		• Arrays are **reference types**: **=** copies the **address**, not the data.
		• **===** compares **identity**, not contents - so **[] === []** is **false**.
		• Edit through one reference and **all references see it**.

Q16 ☐ *Done*
Shallow copy gotcha
Predict: does changing a nested array in the copy affect the original?
	`let original = [[1, 2], [3, 4]];`
	`let copy = [...original];`
	`copy[0].push(99);`
	`console.log(original);  // [[1,2,99],[3,4]]  <- changed too!`
	`let deep = structuredClone(original);  // truly independent`
		• Spread copies only the **top level**.
		• Nested arrays/objects are still **shared references**.
		• Use **structuredClone()** for a **deep copy**.

Q17 ☐ *Done*
Rest parameters + reduce
Write average() that accepts any number of arguments.
	`function average(...nums) {`
	`  return nums.reduce((sum, n) => sum + n, 0) / nums.length;`
	`}`
	`average(10, 20, 30);  // 20`
	`average(5, 10, 15, 20);  // 12.5`
		• **Rest ...nums collects** arguments into a real array.
		• **Spread ...arr expands** an array - same dots, opposite job.
		• Rest must be the **last** parameter.

Section D - Challenge Problems

Combine everything: nested arrays, custom comparison, and a full method chain.

QUESTION	CODE	KEY CONCEPT - WHAT TO REMEMBER

Q18 ☐ *Done*
2D array: access, loop, sum
Read a cell, visit every value, and total the matrix (loops, then nested reduce).
	`let m = [[1, 2, 3], [4, 5, 6], [7, 8, 9]];`
	`m[1][1];  // 5  -> m[row][col]`
	`let sum = 0;`
	`for (let row of m)`
	`  for (let v of row) sum += v;  // 45`
	`// bonus: nested reduce`
	`m.reduce((t, row) => t + row.reduce((s, v) => s + v, 0), 0);`
		• A 2D array is an **array of arrays**: `m[row][col]`.
		• Nested data needs **nested iteration**.
		• Nested reduce: inner reduces each row to a number, outer adds them.
		• Prefer `for...of` over `for...in` for arrays.

Q19 ☐ *Done*
Compare two arrays by values
Write arraysEqual(a, b) for arrays of primitives.
	`function arraysEqual(a, b) {`
	`  if (a.length !== b.length) return false;  // quick exit`
	`  return a.every((v, i) => v === b[i]);  // same order`
	`}`
	`arraysEqual([1, 2, 3], [1, 2, 3]);  // true`
	`arraysEqual([1, 2, 3], [1, 3, 2]);  // false`
		• Length check first, then `every` with the **index** argument.
		• Works only for **primitives**; nested items need recursion.
		• Fixes the `[1,2,3] === [1,2,3]` problem.

Q20 ☐ *Done*
Leaderboard pipeline (method chain)
Keep passed scores, add 5 grace (max 100), sort high to low, take top 3.
	`let scores = [72, 91, 38, 88, 45, 99, 67, 83, 29];`
	`let top3 = scores`
	`  .filter(s => s >= 40)  // 1. keep passed`
	`  .map(s => Math.min(s + 5, 100))  // 2. grace, cap at 100`
	`  .sort((a, b) => b - a)  // 3. high -> low`
	`  .slice(0, 3);  // 4. top three`
	`// [100, 96, 93]  scores is untouched`
		• **Method chaining**: each step returns a new array for the next.
		• `sort` is safe here because `map` already created a **new** array.
		• `Math.min(x, 100)` **caps** a value.
		• Pattern: **validate -> transform -> order -> select**.

Section E - Cheat Sheet

Revise these two tables right before an interview or exam.

MUTATES the original array	Does NOT mutate (returns something new)
`push` `pop` `shift` `unshift` `splice` `sort` `reverse` `fill` `length = n` index assignment `arr[i] = x`	`slice` `map` `filter` `concat` `join` `includes` `indexOf` `find` `findIndex` `some` `every` `reduce` `at` spread `[...arr]` `toSorted`

I WANT TO...	USE	REMEMBER
Do something for each item (log, DOM)	`forEach`	No return value
Transform every item	`map`	Same length, new array
Keep only some items	`filter`	Shorter array or []
Get the first matching item / its index	`find / findIndex`	undefined / -1 if none
Check if any / all items pass a test	`some / every`	Returns true or false
Combine everything into one value	`reduce`	Always give an initial value
Copy or extract a part	`slice(start, end)`	End excluded, no mutation
Add / remove / replace in the middle	`splice(start, count, ...items)`	Mutates the original
Sort numbers correctly	`sort((a, b) => a - b)`	Mutates - copy first
Copy / merge / add without mutating	`[...a, ...b, item]`	Shallow copy only
Accept any number of function arguments	`function f(...args)`	Rest -> real array